Global Analytics Engine - Project Report

1. Executive Overview

Global Analytics Engine is an SRE-focused full-stack observability dashboard that simulates global infrastructure telemetry and presents near real-time operational health for multiple regions.

- Primary objective: demonstrate practical SRE concepts using a working portfolio system.
- Architecture pattern: React frontend + FastAPI backend with cloud-ready deployment workflow.
- SRE focus: Four Golden Signals (Latency, Traffic, Errors, Saturation) plus trend prediction.
- Current maturity: production-aware prototype with local and cloud deployment path.

2. Problem Statement and Goals

Distributed users experience different performance characteristics by geography. The project addresses this by creating a telemetry simulation engine and a global NOC-style dashboard.

- Visualize region-specific latency and reliability posture in a single pane of glass.
- Exercise fault-tolerance and failover reasoning through chaos-testing simulations.
- Showcase end-to-end engineering: backend APIs, frontend UX, containerization, and cloud operations.

3. System Architecture

3.1 Backend Layer (FastAPI)

- Framework: FastAPI (Python 3.11), served via Uvicorn-compatible entry flow.
- Endpoints: `GET /api/health`, `GET /healthz`, and root static serving at `GET /`.
- CORS middleware enabled for cross-origin frontend integration in development.
- Telemetry generation includes randomized signal noise per region (Dublin, New York, Tokyo).
- Intelligence layer classifies trend state: calibrating, stable, unstable, degraded_risk.

3.2 Frontend Layer (React + Vite)

- Stack: React, Axios, Recharts, Tailwind CSS, lucide-react.
- Polling model: every 3000 ms to simulate low-cost real-time stream updates.
- Data transformation pipeline rewires nested API payload into chart/card-friendly state.
- UI modules: regional status cards + intelligence badges + real-time latency chart.
- Operational styling: dark NOC theme and severity-oriented visual feedback.

3.3 Data Flow

- Client requests `/api/health` at fixed heartbeat interval.

- Backend emits per-region telemetry object with four golden signal metrics.
- Frontend normalizes response and updates card-grid and chart in one state update cycle.
- Users observe current risk state and comparative latency posture by region.

4. Key Features Implemented

- Global region telemetry simulation with differentiated base latency profiles.
- Trend intelligence against moving history window for anomaly-like classification.
- Golden signal surfacing for latency, traffic, errors, and saturation.
- Health endpoint (`/healthz`) for cloud load balancer probe compatibility.
- Containerization pathway via multi-stage Docker design for production use.
- Cloud operations narrative including Azure Front Door routing and failover intent.

5. DevOps, Deployment, and SRE Practices

- Container build pipeline references ACR cloud build (`az acr build`).
- App Service runtime hardening includes `WEBSITES_PORT=8000` configuration.
- Global traffic strategy uses Azure Front Door with latency-based routing and probes.
- Chaos testing scenario validates reroute behavior under simulated regional outages.
- Incident narrative demonstrates post-mortem discipline and resilience validation mindset.

6. 7-Day Execution Summary

- Day 1: FastAPI baseline and latency generation foundation.
- Day 2: Frontend-backend integration with CORS and polling.
- Day 3: Recharts observability UI and Dockerization improvements.
- Day 4: Cloud deployment instrumentation and health-check integration.
- Day 5: Multi-region architecture and Front Door routing model.
- Day 6: Chaos engineering and failover verification drills.
- Day 7: Incident post-mortem and documentation closure.

7. Current Limitations and Risks

- Telemetry is simulated, not yet sourced from live probe infrastructure.
- Frontend polling is fixed interval; no adaptive sampling/backoff strategy yet.
- No persistent storage layer for historical analytics or long-range trend analysis.
- No authentication/authorization model for dashboard access in shared environments.
- Potential secret-handling risk if credentials are embedded in repository remotes.

8. Recommended Next Steps

- Externalize environment configuration with clear `.env` strategy and secret vaulting.

- Integrate live telemetry ingestion (e.g., probes, synthetic checks, or agent data).
- Introduce alerting thresholds and event timelines for incident replay.
- Persist metrics in time-series storage for historical dashboards and forecasting.
- Add test suites: backend endpoint tests and frontend rendering/data contract tests.
- Add CI checks for linting, test execution, and container image verification.

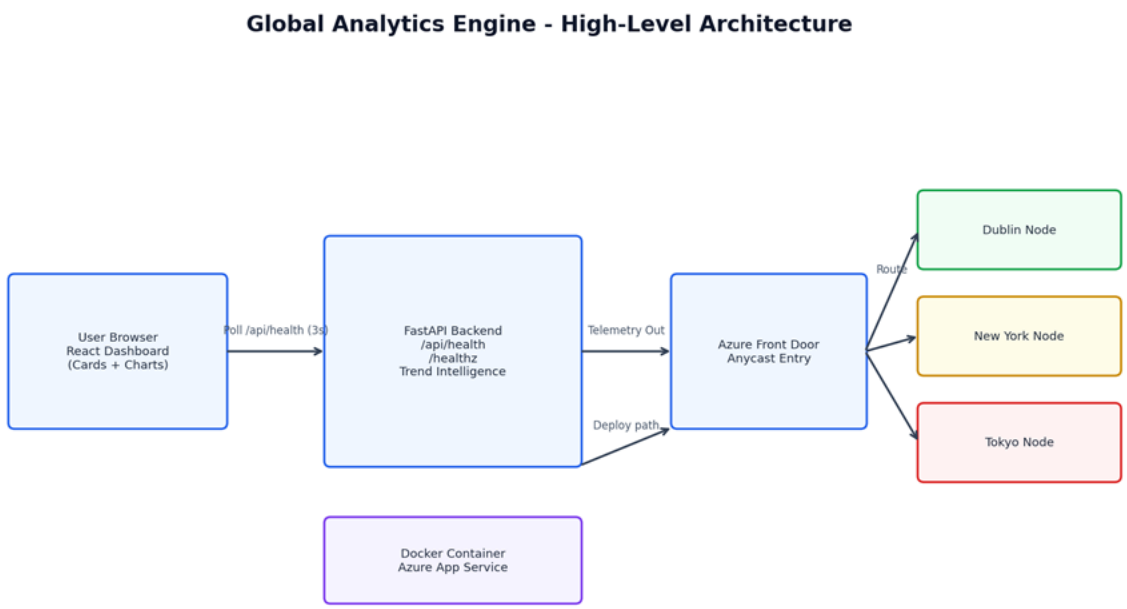
9. Portfolio and Business Value

The project demonstrates practical reliability engineering fundamentals with a clear end-to-end narrative: build, observe, scale, fail, recover, and document. As a portfolio artifact, it effectively communicates both implementation capability and operational thinking.

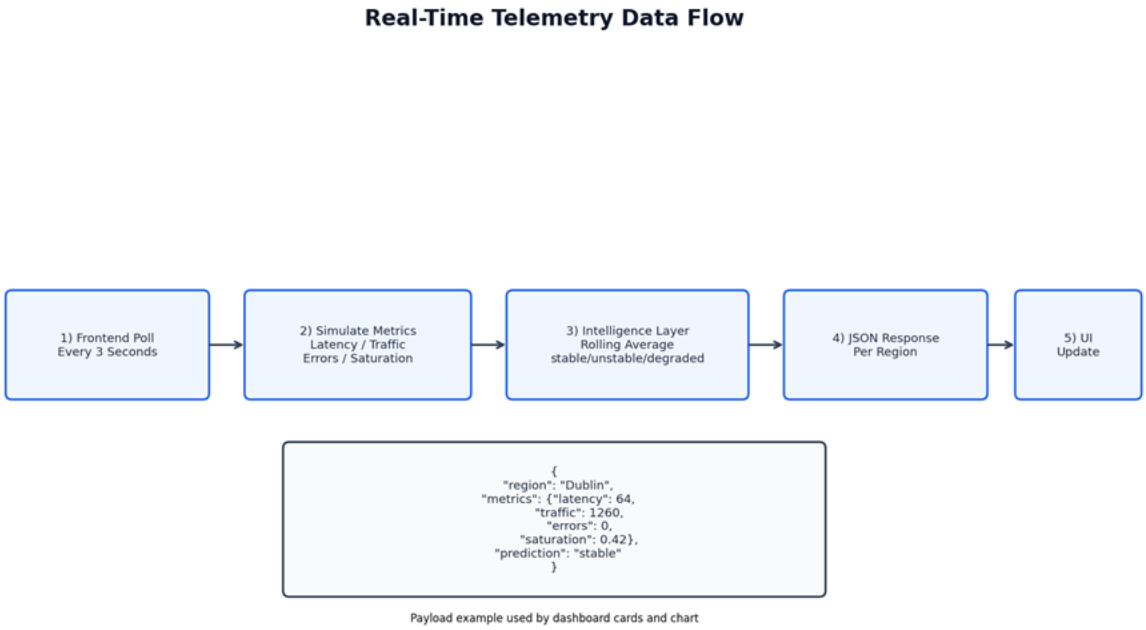
10. Visual Artifacts

The following visuals provide a quick understanding of architecture, telemetry flow, and an in-project screenshot.

10.1 High-Level Architecture Diagram



10.2 Telemetry Data Flow Diagram



10.3 Dashboard Screenshot

global-noc-shweta-bma3g4a3a4bwa4dr.z02.azurefd.net

Global Infrastructure NOC

Dublin

STABLE

Latency: 38ms

Traffic: 835 RPS

Saturation: 31%

New York

STABLE

Latency: 102ms

Traffic: 1111 RPS

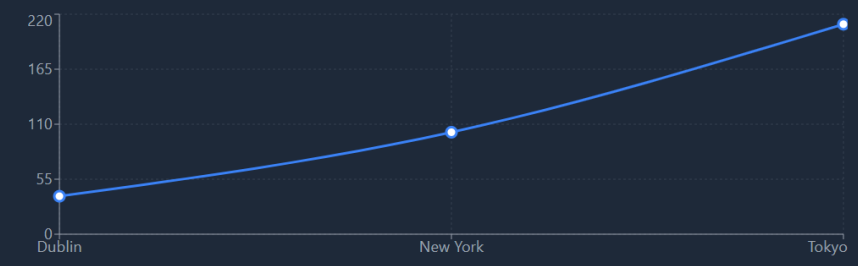
Saturation: 23%

Tokyo

STABLE

Latency: 210ms

Traffic: 1487 RPS

Saturation: 17%

Real-time Latency Stream (ms)